AI for Lawyers: Vibe Coding Your Legal Practice

June 20, 2026

1 Who This Guide Is For

You picked this up for one of a few reasons. Maybe a colleague mentioned AI at lunch and you nodded along. Maybe a client asked whether your firm uses AI and you changed the subject. Maybe you have been watching the headlines, Harvey at $11 billion, Legora at $5.55 billion, and wondering whether you are falling behind.

You are not. But you are right to pay attention.

This guide is for the practising lawyer who wants to understand AI well enough to use it, and maybe even build with it, without going back to computer science school. No coding background is required. A browser, a Gmail account, and curiosity will take you further than you expect.

By the end, you will know how to get free AI API keys, set up a vibe coding environment on your own machine, and build simple legal tools: a contract review assistant, a tabular contract analysis app, a legal research agent. You will also understand the risks, confidentiality, hallucination, professional responsibility, and how to keep those risks under control.

This is not a software manual. It is a practical guide written by someone who has done this work and explained it to lawyers who thought they could not.

1.1 What You Will Be Able to Do

- Get free API keys from Google AI Studio and OpenRouter.
- Set up OpenCode, a free AI coding agent, on Mac or Windows.
- Build a web app that lets you upload a PDF contract and ask questions about it.
- Build a tool that extracts data from multiple contracts into a table.
- Build a legal research assistant that drafts memos with citations.
- Understand when to use free tools, when to pay, and when to keep data off the internet entirely.

If none of those sentences made sense yet, that is fine. They will.

2 What “Vibe Coding” Means for Lawyers

Here is the core idea in one sentence: you describe what you want in plain English, and AI builds it.

That is vibe coding. Not a framework. Not a brand. A way of building software by talking to a machine the way you talk to a colleague, except the colleague writes code at the speed of light and never needs coffee.

The term has been around since early 2025, and by 2026 it is how thousands of non-technical people build working apps every day. James Montemagno, a developer at Microsoft, gave GitHub Copilot a CSV file of his podcast metrics and asked it to “create a beautiful website that helps visualize, search, finds topics, and more.” Five minutes later he had a working dashboard. He fixed a bug by typing “the numbers do not seem correct.” He deployed it by asking for a GitHub Pages workflow. He never opened a terminal.

You already have the skill that matters most. Lawyers draft instructions for a living. You write briefs that tell a court what to decide. You write contracts that tell parties what to do. You write memos that tell a partner what the law says. Vibe coding is the same muscle, pointed at a different audience.

2.1 Why This Differs from Learning to Program

Learning to code traditionally means learning syntax, data structures, debugging, version control, and deployment. Months of study before you build anything useful.

Vibe coding inverts that. You start with the outcome. The AI handles the syntax. You review the result. If it is wrong, you describe what is wrong in plain English and the AI adjusts. The loop is minutes, not months.

This does not mean the output is always correct. It means you can get to "close enough to evaluate" fast, and then iterate. That is a different skill from programming, and for most lawyers, a more useful one.

2.2 The Lawyer's Superpower

The best vibe coders are not the best programmers. They are the best describers. They can articulate what they want, spot what is wrong, and explain the gap.

You do this every day. When you redline a contract, you are comparing an output against an expectation and marking the differences. When you review a junior associate's draft, you are doing quality assurance. When you tell an assistant "I need a summary of the termination clauses, not the whole agreement," you are scoping a task.

Vibe coding rewards exactly those instincts.

3 The Legal AI Landscape in 2026

The legal AI market in 2026 is large, fast-moving, and confusing. Two companies, Harvey and Legora, raised a combined $750 million in fifteen days in March 2026. Harvey sits at an $11 billion valuation. Legora at $5.55 billion. Both are used by major firms. Both are expensive.

But here is what the headlines miss: the free tools available today can replicate a surprising amount of what those platforms do, if you know how to use them.

3.1 From Autocomplete to Agentic Workflows

The first wave of legal AI was autocomplete. Microsoft Copilot in Word would finish your sentence. Useful, but limited.

The current wave is agentic. Instead of completing a sentence, the AI runs a multi-step workflow. You give it a folder of contracts. It classifies them, extracts key clauses, flags risks, and produces a summary report. Harvey reported processing over 400,000 agentic queries per day by early 2026. Legora's Tabular Review turns large document sets into interactive comparison grids.

These are real productivity gains. They are also, in many cases, replicable with free tools and a bit of guidance. That is what this book teaches.

3.2 Why BigLaw Tools Cost €50k Per Year

Enterprise legal AI platforms charge for integration, support, security, compliance, and the comfort of a vendor relationship. Harvey and Legora are built for firms that need single sign-on, audit trails, firm-wide deployment, and a contract they can show a regulator.

If you are a solo practitioner or a small firm, you do not need most of that. You need the AI to work, the output to be reliable, and the data to stay safe. Free and open-source tools can deliver all three, with some trade-offs in convenience.

This guide will show you where the free tools match the paid ones, where they fall short, and how to decide what is right for your practice.

3.3 Ethical and Professional Responsibility Considerations

Using AI in legal work is not just a technical question. It is an ethical one.

Confidentiality. When you send a document to a free public API, that data leaves your device. It may be processed on servers you do not control, in jurisdictions you have not agreed to. For sensitive

client matters, this is not a theoretical risk. It is a professional responsibility issue. We will cover how to handle this, including running AI models locally so data never leaves your machine.

Hallucination. AI models can generate confident-sounding legal analysis that is wrong. Case names that do not exist. Statutes that were never enacted. A lawyer who relies on AI output without verification is still responsible for the result. Every chapter in this book treats verification as a non-negotiable step.

Bar guidance. Regulators are paying attention. The SRA, state bar associations, and courts are issuing guidance on AI use. Some jurisdictions now require disclosure of AI-assisted work product. We will flag the key obligations as they arise.

The human stays responsible. AI can draft, summarise, extract, compare, and propose. The lawyer decides whether the result is accurate, complete, ethical, and fit for the client. That division of labour does not change because the tool gets more capable.

This guide takes those obligations seriously. Every tutorial includes a section on what can go wrong and how to protect your clients and yourself.

4 What Comes Next

The book is organised in six parts. Part 1 helps you understand the legal AI landscape and the tools available. Part 2 walks you through getting free API keys. Part 3 sets up your vibe coding environment. Part 4 is where you build: four tutorials, each producing a working legal tool. Part 5 covers advanced workflows, prompt libraries, and security. Part 6 is a quick-reference appendix.

You do not have to read it front to back. But the tutorials build on each other, so if you are new to this, starting at the beginning will serve you well.

Let us get started.

5 Chapter 1: The legal AI feature set

Imagine a partner drops a folder of 200 contracts on your desk at 4pm on Friday. “I need a summary of all the change-of-control clauses by Monday.” Two years ago, that meant a weekend lost to highlighters and sticky notes. Today, an AI platform can do it in twenty minutes.

The question is not whether AI can handle that work. It can. The question is whether you need to pay enterprise prices to get it done.

This chapter is not a sales pitch for either platform. It is a practical overview of what each one does well, what each one does not do, and how their features map to free or low-cost alternatives that solo and small-firm lawyers can use today.

5.1 Harvey’s core features

Harvey was founded in 2022 by Winston Weinberg, a former securities litigator, and Gabriel Pereyra, a former research scientist at Google DeepMind. By early 2026, it sat at an $11 billion valuation, reported $190 million in annual recurring revenue, and was used by more than 100,000 lawyers across roughly 1,300 organisations. It crossed $100 million in ARR in August 2025 alone.

The platform processes over 400,000 agentic queries per day. That is not a marketing projection. It is production usage.

5.1.1 Document Q&A and analysis

Harvey’s strongest capability, and the one most relevant to this guide, is document analysis. In the 2025 VLAIR benchmark study, Harvey scored 94.8% accuracy on document Q&A tasks, outperforming human lawyers in several categories. You upload a contract or a set of contracts, ask a question in plain English, and the system returns a cited answer.

A typical prompt might be: “What are the termination clauses in this agreement, and are they more favourable to the licensee or the licensor?” Harvey reads the document, finds the relevant sections, and answers with references.

This is the capability we will replicate in Tutorial 1 with free tools.

5.1.2 Vault

Vault is Harvey’s document repository. It lets firms upload up to 100,000 documents into a collaborative, spreadsheet-like workspace. Lawyers can query across the entire repository at once. Instead of

asking about one contract, you can ask: “Across all our NDAs from 2024, which ones have unilateral termination rights?”

For firms managing large contract portfolios, this is a significant step up from folder-based document management. For solo practitioners with a few hundred contracts, a well-organised folder and a free AI tool can get you surprisingly close.

5.1.3 Workflows

Harvey’s Workflows feature lets firms chain multiple AI steps into a single automated process. A due diligence workflow might: classify each document by type, extract key clauses, flag risks, and produce a summary report. The user triggers the workflow; the AI runs the chain.

By early 2026, more than 25,000 custom workflows had been built by Harvey users. This is where the platform moves from “useful assistant” to “operational infrastructure.”

In Part 4 of this guide, we will build a simplified version of this using OpenCode and free AI models.

5.1.4 Agent Builder

Agent Builder lets firms create custom AI agents for specific practice areas. A litigation team might build an agent that specialises in privilege review. A corporate team might build one for M&A due diligence. Each agent can be given custom instructions, access to specific document sets, and a defined scope of work.

This is Harvey’s most powerful feature and the hardest to replicate with free tools. It is also the one most solo lawyers will not need.

5.1.5 Integrations and ecosystem

Harvey has a content partnership with LexisNexis, giving it access to statutes, case law, and citations directly within the platform. In early 2026 it announced a Microsoft 365 Copilot integration, allowing lawyers to access Harvey’s capabilities from within Word and Teams.

These integrations matter because they reduce the number of tools a lawyer has to switch between. They also raise the cost. Harvey does not publish pricing; industry reports suggest indicative costs around $4,000 per user per year for a 10-user team, with six-figure annual contracts for mid-size firms.

5.2 Legora’s core features

Legora, formerly known as Leya, is a Swedish company that reached a $5.55 billion valuation in March 2026. It reported $100+ million in annual recurring revenue, serving over 1,000 firms and in-house teams across more than 50 markets. Its customers include Bird & Bird, Cleary Gottlieb, White & Case, Linkers, Deloitte, Dentons, and Goodwin.

Where Harvey leads on raw scale, Legora is praised for its collaborative workflow design and its strength in multi-jurisdictional work. It was built for international practice from day one.

5.2.1 Tabular review

Legora’s signature feature is Tabular Review. You upload a folder of contracts. The system turns them into an interactive grid: each document is a row, and you define custom prompts that generate columns. One column might extract the governing law clause. Another might pull the termination provisions. A third might flag whether the liability cap exceeds a threshold.

The result is a structured comparison across hundreds or thousands of documents, viewable and exportable as a spreadsheet. For M&A due diligence or large portfolio reviews, this is the feature that makes Legora stand out.

We will build a simplified version of this in Tutorial 2.

5.2.2 Legal AI assistant

Legora’s AI Assistant works within the documents you provide. It handles multi-language documents, provides answers with citations, and maintains context across a conversation. It is designed to reduce hallucinations by grounding responses in the source material rather than relying on the model’s general knowledge.

5.2.3 Microsoft Word add-in

Legora integrates directly into Microsoft Word, letting lawyers review, comment on, and mark up documents from within the add-in. This is a meaningful design choice. Lawyers who live in Word do not have to leave it to use AI. The add-in acts as a second pair of eyes during drafting and review.

5.2.4 Advanced legal research

Legora's research capabilities go beyond document-level Q&A. The system can access trusted legal databases, surface relevant precedents, verify citations, and interpret complex legal language. In May 2026, Legora acquired Qura to build out AI-native legal research, signalling a deeper investment in this area.

For lawyers who rely on up-to-date case law and regulatory guidance, this is a significant capability. Free alternatives exist, though they require more manual effort.

5.2.5 Team collaboration

Legora supports shared projects, prompt libraries that teams can build and reuse, and multi-user access to the same document sets. For firms that want AI capabilities embedded in their team workflow, not just available to individual users, this is a core part of the value proposition.

5.2.6 Pricing

Legora starts at approximately $3,000 per user per year, with a minimum 10-seat commitment. That puts the minimum annual contract at $30,000. Like Harvey, pricing is negotiated based on firm size and modules selected. Legora's implementation includes multi-language and multi-jurisdiction configuration that domestic-only firms may not need.

5.2.7 Integrations

Legora integrates with Microsoft Word and document management systems. It does not yet have the breadth of ecosystem partnerships that Harvey offers, but its European-first architecture gives it an edge on data sovereignty and GDPR compliance that matters to international firms.

5.3 The feature map we will replicate

The rest of this guide is built around a simple premise: most of what Harvey and Legora do at the document level can be replicated with free tools, if you know how to prompt and where to look.

Feature	Paid equivalent	Free tool equivalent
Document Q&A	Harvey	OpenCode + Gemini, Tutorial 1
Tabular contract review	Legora Tabular Review	OpenCode + Llama 4, Tutorial 2
Legal research assistant	Legora Research, Harvey + LexisNexis	OpenCode + DeepSeek R1, Tutorial 3
Contract drafting agent	Copilot Studio	Copilot Studio free tier, Tutorial 4
Due diligence workflow	Harvey Workflows	OpenCode + Gemini 2.5 Pro, Chapter 12
Prompt library	Legora team prompts	Plain text file or Notion, Chapter 13

This is not a perfect mapping. The paid platforms offer security, support, compliance, and scale that free tools cannot match. But for a solo practitioner or a small firm testing the waters, the free tools are enough to build working legal applications. That is what the next three parts of this guide walk through, step by step.

The chapters that follow will show you how to get free API keys, set up a vibe coding environment, and build each of the tools in the right-hand column. You will not need Harvey's budget. You will need patience, a willingness to iterate, and the lawyer's instinct for spotting when something is wrong.

6 Chapter 2: Microsoft Copilot: your free starting point

A partner dictates a client letter into Word on the train home. By the time she reaches the station, Copilot has drafted a polished first version. She edits it on her phone and sends it before dinner. No dictation software, no assistant, no retyping.

That is what Microsoft Copilot does well. It handles the administrative work that surrounds legal practice: emails, documents, meetings, data. It is not built for the work that is legal practice: legal research, contract analysis, case law reasoning.

This chapter covers what Copilot does well for lawyers, where it falls short, and when to reach for something more capable.

6.1 What you already have

Copilot for Microsoft 365 costs $30 per user per month as an add-on to existing M365 business licenses. For a 25-lawyer firm, that is $750 per month. Many firms already have it. If yours does, you have a starting point that requires no new software, no API keys, and no installation.

Here is what Copilot does in each Microsoft 365 application:

6.1.1 Word + Copilot

Copilot can draft documents from prompts, summarise long documents, rewrite sections, and suggest edits. For lawyers, this means first drafts of memos, client letters, and basic legal documents without leaving Word.

A typical prompt: “Draft a client update letter summarising the status of the due diligence process for the Meridian acquisition. Include a section on outstanding items and next steps. Use a professional tone.” Copilot produces a draft. You review, edit, and send.

It does not know your jurisdiction’s rules. It does not know your judge’s preferences. It does not have access to your firm’s clause library. For general document creation, it saves 15 to 30 minutes per document. For anything that requires legal precision, the final review is entirely yours.

6.1.2 Outlook + Copilot

This is where many lawyers see the fastest return. Copilot summarises email threads, drafts replies, prioritises the inbox, and flags action items. For a partner managing 200 emails a day, the time savings are immediate.

“You have 47 unread emails. Three require action today. Here is a summary of each.” That is not a future capability. It is what Copilot does now.

6.1.3 Teams + Copilot

After a meeting, Copilot generates a summary, captures action items, and produces follow-up communications from the transcript. For firms that run client meetings and case conferences through Teams, the automatic summary eliminates the note-taking burden.

The quality of the summary depends on the quality of the audio and the clarity of the speakers. But even an imperfect summary is faster than writing one from scratch.

6.1.4 Excel + Copilot

Copilot can analyse data, create formulae, generate charts, and summarise spreadsheets. For litigation damages calculations, billing analysis, or financial discovery work, it turns a task that might take an hour into one that takes ten minutes.

6.1.5 PowerPoint + Copilot

Copilot creates presentations from documents or prompts. Useful for client pitches, partner meetings, and CLE sessions. The output is a starting point, not a finished product, but it removes the blank-page problem.

6.2 Legal use cases step-by-step

Microsoft’s legal scenario library documents several use cases that law firms have deployed in production. Here are the most relevant ones, with practical prompt examples.

6.2.1 Contract review in Word

Open a contract in Word. Activate Copilot. Use a prompt like:

"Review this contract and identify any clauses that deviate from standard market practice for a commercial lease agreement. Focus on the rent review, break clause, and repair obligations."

Copilot will flag clauses that appear unusual. It will not tell you what the market standard is. That part requires your judgement or a dedicated legal AI with market data. But the first-pass review, identifying what deserves a closer look, is genuinely useful.

6.2.2 Research memo drafting in Word

"Write a first draft of a research memo on the enforceability of non-compete clauses under UK employment law. Cover the current legal framework, recent case law, and practical considerations for advising a client."

Produces a structured draft with headings and body text. The citations are the problem. Copilot does not have access to legal databases. Any case names or statutes it generates must be verified independently. This is not optional. It is a professional responsibility requirement.

6.2.3 Deposition prep in Teams

After a deposition is recorded in Teams, use Copilot to generate a summary:

"Summarise this deposition transcript. Identify the key admissions made by the witness and any inconsistencies with their prior statements."

The summary is a starting point for your preparation. It is not a substitute for reading the transcript. But it lets you focus your reading on the passages that matter.

6.2.4 Due diligence checklist in Excel

"In this spreadsheet, I have a list of target companies. For each one, generate a due diligence checklist covering corporate governance, material contracts, intellectual property, employment matters, and regulatory compliance. Use columns for each category."

Copilot populates the table. You review, adjust for the specific transaction, and distribute to the team.

6.3 Limits of Copilot and when to go further

Copilot is a productivity layer, not a legal layer. It is excellent at the tasks that surround legal work: emails, documents, meetings, data. It is not built for the tasks that are legal work: legal research, contract analysis, case law reasoning, regulatory compliance.

The distinction matters. Here is a practical breakdown:

Task	Copilot sufficient?	Go further when…
Email triage and replies	Yes	Never: Copilot excels here
Meeting summaries	Yes	Never: Teams Copilot is reliable
Basic document drafting	Yes	The document requires legal precision or jurisdiction-specific knowledge
Client letters and updates	Yes	The content involves substantive legal advice
Contract review	Partially	You need market benchmarks or clause-by-clause redlining
Legal research	No	You need verified case law, statutes, or citations
Case law analysis	No	Always: Copilot does not have access to legal databases
Damages calculations in Excel	Yes	The analysis is straightforward

Court filing preparation	No	Court rules and formatting require jurisdiction-specific knowledge
Regulatory compliance review	No	Regulatory databases and current guidance are essential

6.4 Security and data residency

One area where Copilot has a genuine advantage over standalone AI tools is data security. Copilot operates within your Microsoft 365 tenant. It inherits your existing permissions, retention policies, and compliance configurations. Prompts and responses stay within your tenant. Your data is not used to model training.

For firms already trusting Microsoft 365 with client data, which is virtually every firm, Copilot does not introduce a new security risk. The key requirement is that your Microsoft 365 security policies are properly configured before enabling Copilot. If your tenant still has default settings and open sharing policies, fix those first.

Microsoft reports that Copilot complies with SOC 2, ISO 27001, GDPR, and HIPAA through the broader Microsoft 365 compliance framework. For firms with regulatory obligations, this existing compliance infrastructure is a meaningful advantage over consumer AI tools.

6.5 The billable hour question

Copilot creates an uncomfortable question for firms that bill by the hour. If AI does in ten minutes what used to take an hour, the hours-based revenue for that work drops. At $30 per user per month, Copilot pays for itself if it saves 45 minutes per day per lawyer. At typical billing rates, that recovered time is worth $4,500 to $7,500 per month per lawyer.

Some firms are responding by moving toward value-based pricing for routine work. Others are simply absorbing the efficiency gain as a competitive advantage. Either way, the firms that adopt these tools early are better positioned than the firms that wait.

6.6 The verdict on Copilot

Use Copilot for everything it handles well: email, meetings, basic documents, data analysis, and the administrative work that surrounds legal practice. Do not use it for substantive legal work that requires verified legal knowledge, jurisdiction-specific reasoning, or reliable citations.

For that work, you need the tools we will set up in the next two parts. Start with Copilot because it is already in your firm. Then build from there.

Note: Statistics and pricing figures in this chapter are drawn from industry reports and vendor claims referenced in the outline. TODO: verify against primary sources before publication.

7 Chapter 3: Google AI Studio: your first free API key

Before you can build anything, you need a key. Not a physical key. An API key: a string of characters that lets your tools talk to an AI model over the internet.

Think of it like a password for a service. You give the password to an app you trust, and the app uses it to access the AI on your behalf. Without the key, the AI will not respond. With it, you can send prompts and receive answers, build tools, and automate work.

This chapter walks you through getting your first key from Google AI Studio. It takes about two minutes.

7.1 What an API key unlocks

Google AI Studio gives you access to the Gemini family of models. The free tier includes Gemini 2.5 Pro, which has a context window of one million tokens. That is enough to feed in a lengthy contract and ask questions about it without hitting a size limit.

A token is roughly a word or a piece of a word. One million tokens is approximately 750,000 words, or about 1,500 pages of text. For most legal documents, that is more than enough.

The free tier has rate limits: a cap on how many requests you can send per minute and per day. For building and testing tools, the free tier is sufficient. If you later need higher limits, you can upgrade to a paid plan. But start free.

7.2 Step-by-step: getting your Gemini key

1. Open your browser and go to https://aistudio.google.com.
2. Sign in with your Gmail account. If you do not have one, create one. It is free.
3. On the left sidebar, click "Get API key." If you do not see it, look for a button labelled "Create API key in new project."
4. Google Cloud will create a project for you and generate a key. It will look something like `AIzaSyD...` followed by a long string of random characters.
5. Copy the key immediately. Store it in a password manager. If you lose it, you can generate a new one, but you will need to update every tool that used the old one.
6. Check your free quota limits. In Google AI Studio, navigate to the pricing or quota page to see how many requests per minute and per day the free tier allows. As of 2026, the free tier of Gemini 2.5 Pro allows a generous number of requests per day, enough for development and testing.

That is it. You now have a key. Do not share it. Do not paste it into public websites or email it to colleagues. Treat it like a password.

7.3 Testing your key (no code required)

Google AI Studio has a built-in playground where you can test your key without writing a single line of code.

1. In Google AI Studio, click "Create new" or "Chat" from the left sidebar.
2. A chat interface appears. In the model dropdown, select "Gemini 2.5 Pro."
3. In the system instructions or the chat box, type a legal prompt. For example:
   "Summarise the key termination clauses in the following contract excerpt. Identify any unusual provisions."
4. Paste a short contract clause below the prompt.
5. Press Enter. The model will respond with a summary.

If it works, your key is active and the model is responding. If you get an error, check that you are signed in, that your key is valid, and that you have not exceeded your rate limit.

7.4 Understanding rate limits and the free tier

Every free API has limits. Google's free tier for Gemini allows a certain number of requests per minute and per day. If you exceed the per-minute limit, you will get a brief error and can retry moments later. If you exceed the daily limit, you will need to wait until the next day or upgrade.

For the work in this guide, building and testing legal tools, the free tier is enough. You are not running a production service with thousands of users. You are a lawyer building a tool for yourself or your firm. The free tier supports that.

If you find yourself hitting limits regularly, two options: upgrade to Google's paid tier, or use OpenRouter (covered in the next chapter) to access additional models with their own free quotas.

8 Chapter 4: OpenRouter: one key, 75+ free models

Google's key gives you access to Gemini. OpenRouter gives you access to almost everything else, through a single key and a single interface.

8.1 Why OpenRouter changes everything

OpenRouter is a unified API gateway. Instead of signing up for Google, Anthropic, Meta, and DeepSeek separately, you create one OpenRouter account, get one key, and use it to access models from all of them.

As of 2026, OpenRouter offers access to over 75 models. Many of them are free. No credit card is required to get started. You create an account, generate a key, and begin.

The free models are not stripped-down demos. They are the same model weights as the paid versions, subject to rate limits and slightly higher latency during peak hours. For building and testing legal tools, they are more than capable.

8.2 Step-by-step: creating your OpenRouter key

1. Go to https://openrouter.ai.
2. Click “Sign In.” You can sign up with a Google account or an email address. No credit card is required.
3. Once logged in, click your profile icon in the top right corner. Select “Keys” from the dropdown menu.
4. Click “Create Key.” Give it a name that will help you remember its purpose. “Legal-tools” works fine.
5. Click “Create.” OpenRouter generates a key that looks like `sk-or-v1-...` followed by a long string.
6. Copy the key immediately. OpenRouter only shows it once. If you close the window without copying it, you will need to generate a new key.
7. Store the key in a password manager or a `.env` file on your machine. Never hardcode it in a script or share it in an email.

You now have access to the full OpenRouter catalogue. To stay on the free tier, you do not need to add any credits. Free models work with a zero balance.

8.3 Choosing the right free model for legal work

Not all models are equally good at all tasks. Here is a comparison of the most useful free models on OpenRouter for legal work as of 2026:

Model	Best for	Context window	Notes
DeepSeek R1	Reasoning, analysis, complex questions	64K	Slower but thorough. Good for contract analysis.
DeepSeek Chat V3	General chat, drafting, summarisation	64K	Faster than R1. Good all-rounder.
Llama 4 Maverick	Long documents, multimodal tasks	1M	Best free option for very large documents.
Qwen3 235B	Coding, structured extraction	128K	Strong for building tools that process data.
Gemini 2.5 Flash	Fast responses, large context	1M	Google’s fast model. Good for quick tasks.
Mistral Small 3.1	Writing, balanced tasks	128K	Reliable and fast. Good default choice.
Grok 3 Mini	Fast, lightweight reasoning	131K	Optimised for speed over depth.

8.3.1 When to use a reasoning model

DeepSeek R1 is a reasoning model. It “thinks” before it answers, working through the problem step by step. This makes it slower than other models, but more accurate for complex legal questions.

Use R1 when you are asking the model to analyse a contract, compare clauses, or reason through a legal problem. Use a faster model like Gemini Flash or Mistral Small when you need a quick summary or a draft.

8.3.2 When context window matters

The context window is how much text the model can consider at once. If you are analysing a 200-page lease agreement, you need a model with a large context window. Llama 4 Maverick and Gemini 2.5 Flash both offer one million tokens, enough for the longest contracts you will encounter.

If your document is shorter than about 50 pages, any model on the list will handle it.

8.3.3 A practical starting point

If you are unsure which model to pick, start with DeepSeek Chat V3 for general tasks and switch to DeepSeek R1 when you need deeper analysis. For tools that process long documents, use Llama 4 Maverick or Gemini 2.5 Flash.

You can change models at any time. The key stays the same. You just tell OpenRouter which model to use for each request.

8.4 Keeping your key safe

Your OpenRouter key is a credential. Anyone who has it can use your account, consume your free quota, or run up charges if you have added credits.

Three rules:

1. Store it in a password manager or a `.env` file, never in the code itself.
2. If you accidentally expose it (pushed it to a public repository, pasted it in a chat), regenerate a new key immediately from the OpenRouter dashboard and delete the old one.
3. Add optional headers to your requests (`HTTP-Referer` and `X-Title`) so you can track which project is using your quota in the OpenRouter activity dashboard.

With your Google AI Studio key and your OpenRouter key, you now have access to the most powerful free AI models available. The next part of this guide shows you how to set up the tool that uses these keys to build legal applications.

9 Chapter 5: Installing OpenCode: the free Cursor alternative

You type a sentence in plain English. Three minutes later, you have a working web application. No syntax errors to debug, no Stack Overflow tabs to open, no computer science degree required. That is the promise of OpenCode, and unlike most promises in the AI space, it largely delivers.

OpenCode is an open-source AI coding agent that runs in your terminal. It is not a plugin for VS Code or a web app you sign up for. It is a standalone program that opens a text-based interface, connects to your chosen AI model through the keys you already have, and builds software based on plain English instructions.

As of 2026, OpenCode has over 167,000 GitHub stars. It is MIT licensed, which means it is free to use, modify, and distribute. It supports 75+ AI models across every major provider: Anthropic, OpenAI, Google, OpenRouter, and local models through Ollama.

Most importantly for this guide, it was built around the idea that you should review a plan before any code gets written. That philosophy aligns well with how lawyers already work: think before you act, review before you file.

9.1 What OpenCode is and why it matters

OpenCode is a terminal-first AI coding agent. When you launch it, you get a clean text interface inside your terminal window. You type a request in plain English. OpenCode reads your project files, formulates a plan, and (when you approve) writes code, creates files, runs commands, and tests the result.

It is not the only tool that does this. Claude Code and OpenAI's Codex CLI are alternatives. What makes OpenCode different is three things:

Provider freedom. Claude Code only works with Anthropic models. Codex CLI only works with OpenAI. OpenCode works with 75+ models across every provider. You are not locked in.

Open source. The code is publicly auditable. You can see exactly what it does with your files and your data. For lawyers handling sensitive matters, that transparency is not optional.

Price. The core agent costs nothing. You pay only for API usage to your chosen provider, and only when you use it. There is no subscription, no premium tier, no feature gate.

9.2 Two modes: plan before you build

Every OpenCode session runs in one of two modes, switchable with the Tab key:

Plan mode is read-only. OpenCode analyses your codebase, drafts what it intends to do, and presents the plan for review. It does not touch any files. You read the plan, push back on anything that looks wrong, and refine until you are satisfied.

Build mode is where OpenCode executes. It reads files, writes code, runs shell commands, and makes the changes you approved in Plan mode.

This design solves the most common problem with AI coding agents: they dive into changes you did not ask for and keep going when you try to stop them. Making "plan then execute" a built-in concept, not a prompting trick, is one of OpenCode's better decisions.

There are also three specialised modes: Debug (investigation only), Review (code review only), and Docs (reads and writes documentation files).

9.3 Installation on Mac (step-by-step)

This section assumes no prior command-line experience. If you have never opened Terminal, that is fine.

1. Open Terminal. Press Cmd+Space to open Spotlight, type "Terminal," and press Enter. A white or black window appears.
2. Install Homebrew if you do not have it. Homebrew is a package manager that makes installing developer tools easy. Paste this command and press Enter:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.
sh)"
```

Follow the prompts. It will ask for your Mac password.

3. Install Node.js:

```
brew install node
```

4. Install OpenCode:

```
npm install -g opencode-ai
```

5. Launch OpenCode:

```
opencode
```

On first launch, OpenCode prompts you to configure a provider. Enter your OpenRouter or Gemini API key when asked.

6. Verify it works. At the prompt, type:

```
Explain what you can do in this project.
```

OpenCode should respond. If it does, you are set up.

9.4 Installation on Windows (step-by-step)

1. Download Node.js from https://nodejs.org. Choose the LTS version. Run the installer and accept the defaults.
2. Open PowerShell as Administrator. Right-click the Start button and select "Windows PowerShell (Admin)" or "Terminal (Admin)."
3. Install OpenCode:

```
npm install -g opencode-ai
```

4. Launch it:

```
opencode
```

5. Enter your API key when prompted.
6. Optional: install Windows Terminal from the Microsoft Store for a better experience. It supports tabs, better fonts, and a more readable display than the default PowerShell window.

9.5 First run: the OpenCode interface explained

When you launch OpenCode, you see a terminal interface with a prompt at the bottom. This is where you type your requests.

The basic interaction loop:

1. Type a request. For example: "Build me a simple web app where I can upload a PDF and ask questions about it."
2. OpenCode analyses your project (or creates a new directory if the project does not exist yet).
3. If you are in Plan mode (check the status bar), OpenCode drafts a plan. Read it. Ask questions. Request changes.
4. When you are satisfied, press Tab to switch to Build mode. OpenCode executes the plan.
5. Test the result. If something is wrong, describe the fix. OpenCode adjusts.
6. Save your session. OpenCode persists sessions in SQLite, so you can close the terminal and resume later.

Useful commands:

- `/undo` undoes the last change.

- `/redo` restores an undone change.
- `@` followed by text searches for files in your project.
- `Tab` switches between Plan and Build mode.

9.6 Understanding what OpenCode is good at

OpenCode excels at building web applications from natural language descriptions. Say "build me a contract review tool" and it will scaffold a working app: HTML front end, a backend to handle file uploads, integration with your chosen AI model, and a simple interface for asking questions about uploaded documents.

It is less good at tasks that require deep domain expertise it does not have. It does not know UK contract law. It does not know your firm's precedent database. It does not know the difference between a reasonable and an unreasonable indemnity clause unless you tell it.

That is where your job comes in. You provide the legal context, review the output, and iterate. OpenCode handles the plumbing: the code, the file structure, the API integration, the debugging.

9.7 The honest trade-offs

OpenCode is powerful, but it is not magic. Here is what to expect:

Iteration is the process. The first version will rarely be perfect. You will need to test, describe what is wrong, and ask for changes. This is normal. It is how all software gets built, with or without AI.

Slower than Claude Code. Independent testing by Builder.io found OpenCode is roughly 78% slower than Claude Code on identical tasks using the same model. The extra time comes from OpenCode's LSP integration, which checks the code against a real compiler before committing to changes. For correctness, that trade-off is worth it.

Terminal-only. OpenCode runs in a terminal. There is no graphical interface, no inline code completion, no visual diff viewer. If you prefer a GUI, consider VS Code with GitHub Copilot (covered in the next chapter). But for the work in this guide, the terminal is sufficient.

Configuration required. OpenCode does not work perfectly out of the box. You need to configure your provider, set your API key, and optionally create an `AGENTS.md` file in your project to give OpenCode context about what you are building. The first-time setup takes about five minutes.

10 Chapter 6: Optional: VS Code + GitHub Copilot (free tier)

The blinking cursor in a black terminal window is not everyone's idea of a good time. If you have never used a command line, OpenCode's interface can feel like being handed the controls of a plane mid-flight. This chapter is for you.

VS Code with GitHub Copilot gives you the same AI assistance through a familiar graphical interface. Files in a sidebar. Code in a window. Suggestions that appear as you type. It is less powerful than OpenCode for building entire applications from scratch, but it is more comfortable for many lawyers, and comfort matters when you are learning something new.

10.1 When to use VS Code instead of OpenCode

Choose VS Code if:

- You prefer a graphical interface over a terminal.
- You want to see files, folders, and code in a visual editor.
- You want inline code suggestions as you type (Copilot's autocomplete).
- You are already comfortable with VS Code or another code editor.

Choose OpenCode if:

- You want the most powerful AI coding agent available for free.
- You need access to 75+ models across multiple providers.
- You prefer a terminal workflow.
- You want the Plan/Build mode separation.

You can use both. Many developers run OpenCode in VS Code's integrated terminal, getting the best of both worlds.

10.2 Quick setup

1. Download VS Code from https://code.visualstudio.com. It is free.
2. Open VS Code. Click the Extensions icon on the left sidebar (or press Cmd+Shift+X on Mac, Ctrl+Shift+X on Windows).
3. Search for "GitHub Copilot." Click Install.
4. Sign in with a free GitHub account. If you do not have one, create one at github.com.
5. To connect your OpenRouter key: open VS Code settings, search for "Copilot Chat," and look for the provider settings. You can configure OpenRouter as a custom provider.
6. Open a folder (File → Open Folder) and start working. Copilot will suggest code as you type. Copilot Chat (accessible from the sidebar) lets you describe tasks in plain English, similar to OpenCode.

10.3 What the free tier includes

GitHub Copilot's free tier provides 2,000 code completions per month and a limited number of chat interactions. For the tutorials in this guide, that is enough to get started. If you find yourself hitting limits, you can upgrade to the paid tier ($10/month) or use OpenCode with your OpenRouter key for unlimited access to free models.

10.4 A note on what VS Code cannot do

VS Code with Copilot is an excellent code editor with AI assistance. It is not the same as OpenCode. Copilot suggests code as you type and answers questions in a chat panel. It does not independently plan and execute multi-file projects the way OpenCode does.

For the tutorials in Chapters 8 through 11, OpenCode is the recommended tool. It can take a single plain English request and build an entire working application. Copilot can help you write and edit code, but you will need to do more of the architectural thinking yourself.

Note: Statistics and performance claims in this chapter are drawn from vendor documentation and third-party testing referenced in the outline. TODO: verify against primary sources before publication.

If you are comfortable in a terminal, use OpenCode. If you are not, start with VS Code and migrate to OpenCode when you are ready. Either way, you will be building legal tools by the end of Part 4.

11 Chapter 7: The vibe coding mindset for lawyers

You describe what you want. The AI builds it. You test it. You describe what is wrong. The AI fixes it. That loop, describe, build, test, refine, is the core of vibe coding. It is also, if you think about it, the core of how lawyers have always worked with junior staff, assistants, and colleagues.

The tutorials in Chapters 8 through 11 will give you step-by-step instructions. This chapter explains how to approach those instructions so you get good results instead of frustration.

11.1 Your new workflow

The biggest mistake new vibe coders make is opening the tool and typing "build me a legal app." That is like walking into a meeting with a junior associate and saying "write me a brief." You would never do that. You would give context, constraints, a deadline, and a clear description of the output you need.

The same discipline applies here. Follow this workflow for every tool you build:

Step 1: Write a one-page spec before opening any tool. Describe what you want in plain English. Who will use it? What does it do? What are the inputs and outputs? What does success look like? This spec is not for the AI. It is for you. It forces you to think clearly before you start.

Step 2: Say "tell me your plan, don't build yet." In OpenCode, stay in Plan mode. Let the AI draft what it intends to do. Read the plan carefully. Does it understand what you asked for? Is it missing anything? Is it doing something you did not ask for? This is the review step, and it is the most important one.

Step 3: Adjust the plan until you are satisfied. Ask questions. Request changes. "Add a risk-flagging feature." "The output should be exportable to Word." "Use the Gemini API, not OpenRouter." Refine the plan until it matches your spec.

Step 4: Switch to Build mode and approve. Only when the plan is right do you let OpenCode touch any files. Press Tab to switch to Build mode. Watch what it does. If something goes wrong, use `/undo` and describe the problem.

Step 5: Test, describe what is wrong, iterate one change at a time. The first version will be 70-80% right. Test it. When you find a problem, describe it specifically and ask for one fix at a time. “The upload button does not work on mobile” is better than “fix the interface.”

11.2 Prompt patterns for legal apps

A prompt is what you type to tell OpenCode what to build. The quality of the output depends on the quality of your prompt. Here are three patterns that work well for legal tools.

11.2.1 The “Harvey-style” Q&A prompt template

Use this pattern when you want to build a tool that lets users upload documents and ask questions about them.

```
Build me a web app with the following features:
- Users can upload a PDF document
- Users can type questions about the document in a chat interface
- The app sends the document and the question to the AI
- The AI responds with an answer and cites the relevant section of the document
- The interface is clean and works on both desktop and mobile
- Use [Gemini API / OpenRouter with DeepSeek R1] for the AI backend
- Show me your plan first before building anything.
```

The key elements: what the user does, what the app does, what the AI does, which model to use, and the instruction to plan first.

11.2.2 The “Legora tabular review” extraction prompt template

Use this pattern when you want to build a tool that extracts structured data from multiple documents.

```
Build me a web app with the following features:
- Users can upload multiple PDF contracts at once
- For each contract, extract the following fields: Party Names, Governing Law, Termination
Clause, Liability Cap, Notice Period, Renewal Terms
- Display the results in a table with one row per contract
- Allow the user to download the table as a CSV file
- Use [OpenRouter with Llama 4] for the AI backend
- Show me your plan first before building anything.
```

Notice the specific column definitions. The more precisely you describe what you want extracted, the better the output.

11.2.3 How to give context: jurisdiction, practice area, document type

AI models do not know your practice area unless you tell them. When building a tool, include context in your prompt:

```
This tool is for a UK commercial law practice. It should:
- Use British legal terminology (e.g., "solicitor" not "attorney")
- Default to English law as the governing law
- Reference UK court procedures where relevant
- Format dates as DD/MM/YYYY
```

This context shapes every output the AI generates. Without it, the tool may use American terminology, reference US law, or format things in ways that feel foreign to your practice.

11.3 What to expect (and not expect)

11.3.1 First attempt is rarely perfect

The first version of anything you build will have problems. A button that does not work. A layout that breaks on mobile. An AI response that misses the point. This is normal. It is not a sign that the tool is broken or that you are doing it wrong. It is how software gets built.

The difference between a successful build and a failed one is not the quality of the first attempt. It is the willingness to iterate. Plan for three to five rounds of feedback on every tool you build.

11.3.2 AI hallucinates: never trust legal citations without verification

This bears repeating. When an AI model generates a case name, a statute, or a legal proposition, it may be wrong. Confidently, plausibly, and completely wrong.

Every tool you build for legal work should include a disclaimer that outputs are for informational purposes only and must be verified against primary sources. This is not a limitation of the tool. It is a professional responsibility.

When you test your tools, pay special attention to any legal citations the AI generates. If it references a case, look it up. If it quotes a statute, check the actual text. If the citation does not exist, that is a hallucination, and you need to add a warning to the user.

11.3.3 Version control basics: how to undo

OpenCode has a built-in undo. Type `/undo` to reverse the last change. Type `/redo` to restore it. Use these liberally. If a change makes things worse, undo it and try a different approach.

For more permanent version control, you can initialise a git repository in your project folder. This lets you save snapshots of your project at any point and roll back to a previous version if something goes wrong. OpenCode can do this for you: ask it to "initialise a git repository and make an initial commit."

The rule is simple: revert early, revert often. Do not let a broken state persist because you hope the next fix will resolve it. Go back to the last working version and try again.

12 Chapter 8: Tutorial 1: Contract review assistant (Harvey's document Q&A)

This is the first build. You are going to create a web application that lets you upload a PDF contract, ask questions about it, and get cited answers. It is the free equivalent of Harvey's Document Q&A and Legora's AI Assistant.

Estimated time: 30-45 minutes.

12.1 What we are building

A local web application with three features:

1. A file upload interface for PDF contracts.
2. A chat interface where you type questions about the uploaded contract.
3. AI-generated answers that cite the relevant sections of the document.

The app runs on your machine. Your documents never leave your computer unless you choose to send them to a cloud API. We will cover how to keep your data local in the security section.

12.2 Step-by-step build with OpenCode

1. Create a project folder and open OpenCode:

```
mkdir contract-review
cd contract-review
opencode
```

2. Switch to Plan mode (press Tab if you are in Build mode).
3. Paste this prompt:

```
Build me a simple web app where I can upload a PDF contract and ask questions about it. The
app should have:
- A clean upload page where I can drag and drop a PDF
- A chat interface where I can type questions about the contract
- The AI should answer based on the content of the uploaded PDF
- Answers should include citations referencing the relevant part of the document
- The app should run locally on my machine
- Use the Gemini API for the AI backend
- Show me your plan first. Do not build yet.
```

4. Read the plan. Check for:
   - Does it understand the PDF upload requirement?
   - Does it plan to use the Gemini API as requested?
   - Does it include a chat interface?
   - Does it mention citation of source material?
5. Request adjustments if needed. For example:

```
Add a feature that flags potentially risky clauses (unlimited liability, unilateral
termination, automatic renewal). Highlight these in yellow in the response.
```

6. When you are satisfied with the plan, press Tab to switch to Build mode. OpenCode will create the project files. This takes a few minutes.
7. When OpenCode finishes, it will tell you how to run the app. Typically:

```
npm install
npm run dev
```

8. Open the URL it gives you (usually http://localhost:3000) in your browser.
9. Test it. Upload a short PDF contract (a sample NDA works well). Ask a question: “What are the termination provisions?”
10. Evaluate the response. Is it accurate? Does it cite the right section? Is the interface usable?
11. Iterate. Describe what needs to change. For example:

```
The upload button is too small. Make it a full-width drag-and-drop zone. Also, add a loading
indicator while the PDF is being processed.
```

12.3 Connecting your Gemini key safely

The app needs your Gemini API key to talk to the AI. Here is how to connect it without exposing the key.

On Mac or Linux, create a file called `.env` in your project folder:

```
GEMINI_API_KEY=AIzaSyD...your key here...
```

On Windows, create the same file:

```
GEMINI_API_KEY=AIzaSyD...your key here...
```

The app should be configured to read from this file. If OpenCode did not set this up automatically, ask it to:

```
Update the app to read the Gemini API key from a .env file instead of hardcoding it.
```

Three rules for API key safety:

1. Never paste your API key directly into the code. Always use a `.env` file.
2. Add `.env` to your `.gitignore` file so it never gets uploaded to a public repository. Ask OpenCode to do this: “Add .env to .gitignore.”
3. Never share the `.env` file, email it, or paste it into a chat. If you accidentally expose the key, regenerate a new one from Google AI Studio immediately and update the file.

13 Chapter 9: Tutorial 2: Tabular contract review (Legora’s tabular review)

This tutorial builds a tool that takes multiple PDF contracts, extracts specified data from each one, and displays the results in a table you can download. It is the free equivalent of Legora’s Tabular Review.

Estimated time: 30-45 minutes.

13.1 What we are building

A web application with three features:

1. A multi-file upload interface for PDF contracts.
2. AI extraction of user-defined fields from each contract.
3. A results table, downloadable as CSV, that you can open in Excel.

13.2 Designing your extraction template

Before you prompt OpenCode, decide what data you want extracted. Write it down. For a typical commercial contract review, you might want:

- Party Names
- Governing Law
- Termination Clause summary
- Liability Cap amount
- Notice Period duration
- Renewal Terms

You can customise these to your practice area. A litigation colleague might want different fields: dispute resolution mechanism, jurisdiction, limitation period, indemnification scope.

The key decision is whether to extract all fields in one prompt or one field per prompt. One combined prompt is faster but may miss details. Separate prompts per field are more thorough but take longer. For your first build, use one combined prompt. You can refine later.

13.3 Step-by-step build

1. Create a project folder and open OpenCode:

```
mkdir tabular-review
cd tabular-review
opencode
```

2. Switch to Plan mode.
3. Paste this prompt, customising the fields to your needs:

```
Build a web app to upload multiple PDF contracts. For each contract, extract the following
fields: Party Names, Governing Law, Termination Clause (summarise in 2-3 sentences), Liability
Cap, Notice Period, Renewal Terms. Display results in a table with one row per contract and
one column per field. Allow the user to download the table as a CSV file. Use OpenRouter with
Llama 4 Maverick for the AI backend. Show me your plan first.
```

4. Review the plan. Check that it includes multi-file upload, table display, and CSV export.
5. Request adjustments. For example:

```
Add a progress indicator that shows which contract is being processed. Also, add a column for
the contract file name so I can match rows to source documents.
```

6. Switch to Build mode. Let OpenCode build the app.
7. Run the app and test it with two or three sample contracts.
8. Evaluate the extraction quality. Are the fields populated correctly? Is the termination clause summary accurate? Are there fields the AI left blank or got wrong?
9. Iterate. If the extraction is inconsistent, try breaking it into separate prompts:

```
Instead of extracting all fields at once, process each field with a separate prompt. First
pass: extract Party Names from all contracts. Second pass: extract Governing Law. Continue for
each field.
```

10. Export the results to CSV and open in Excel. Check the formatting. Ask OpenCode to fix any issues with how the data appears.

14 Chapter 10: Tutorial 3: Legal research agent

This tutorial builds a chatbot that takes a legal question, researches it, and drafts a memo with citations. It uses DeepSeek R1, a free reasoning model available through OpenRouter, for deep analysis.

Estimated time: 45-60 minutes.

14.1 What we are building

A web application with three features:

1. A text input where you type a legal question.
2. AI-generated research that searches for relevant information.
3. A formatted memo with citations, exportable as a Word document.

This is the most ambitious tutorial. It involves web search, document generation, and careful prompt engineering. The result is a tool that can draft a first-pass research memo in minutes.

14.2 Connecting to external legal sources (optional)

The quality of the research depends on the sources the AI can access. By default, the model uses its training data, which has a knowledge cutoff and may not include the latest cases or statutes.

You can improve results by connecting to public legal databases:

- EUR-Lex (EU law): https://eur-lex.europa.eu
- CourtListener (US case law): https://www.courtlistener.com
- caselaw.io (various jurisdictions)

The technique is called RAG: Retrieval-Augmented Generation. In plain English, it means the AI searches for relevant documents first, then uses those documents to answer your question instead of relying on memory alone.

For this tutorial, we will use the model's built-in capabilities. In Chapter 12, we will cover more advanced workflows.

14.3 Step-by-step build

1. Create a project folder and open OpenCode:

```
mkdir legal-research
cd legal-research
opencode
```

2. Switch to Plan mode.
3. Paste this prompt:

```
Build a legal research assistant web app. The user types a legal question. The app researches
the question and drafts a memo with the following sections: Question Presented, Brief Answer,
Discussion, and Conclusion. The memo should include citations to relevant case law and
statutes. Allow the user to export the memo as a .docx file. Use DeepSeek R1 via OpenRouter
for the AI backend. Show me your plan first.
```

4. Review the plan. Check that it includes the memo structure, citation formatting, and docx export.
5. Request adjustments:

```
Add a field for the user to specify the jurisdiction (e.g., England and Wales, New York,
Federal). Use this to tailor the research and citations. Also, add a disclaimer that the memo
is a first draft and all citations must be verified.
```

6. Switch to Build mode. Let OpenCode build the app.
7. Run the app and test it with a real question. For example:

   "What are the key factors courts consider when deciding whether to grant an interim injunction in England and Wales?"

8. Evaluate the output. Is the memo well-structured? Are the citations real? (Check them. Every single one.) Is the analysis accurate?
9. Iterate. Common issues and fixes:

```
The citations include case names that sound plausible but do not exist. Add a step where the
model verifies each citation against a legal database before including it in the memo.
The memo is too generic. Ask the model to include specific case references, statutory
provisions, and practical advice for the solicitor handling the matter.
```

15 Chapter 11: Tutorial 4: MS Copilot Studio agent for contract drafting

This tutorial is different from the previous three. Instead of building a web app with OpenCode, you will build a contract drafting agent using Microsoft Copilot Studio. This agent lives inside Microsoft Teams and Word, where your lawyers already work.

No terminal required. No API keys to manage. If your firm has Microsoft 365, you likely have access to Copilot Studio's free tier.

15.1 Using Copilot Studio (free tier)

Microsoft Copilot Studio is a platform for building custom AI agents that integrate with Microsoft 365. The free tier allows you to create agents that can:

- Answer questions based on documents you upload
- Follow custom instructions and topics
- Integrate with Teams and Word
- Use Microsoft's AI infrastructure (no separate API key needed)

This is the right tool when you want AI capabilities embedded in your existing workflow, not in a separate application.

15.2 Building a contract drafting agent

1. Open https://copilot.microsoft.com/studio in your browser. Sign in with your Microsoft 365 account.
2. Click "Create" and select "New agent."
3. Name it "Contract Drafting Assistant."
4. In the instructions field, paste:

```
You are a contract drafting assistant for a UK commercial law practice. Your role is to help
lawyers draft first versions of standard contracts. When asked to draft a contract or clause:
- Use British legal terminology
- Default to English law
- Follow the firm's style guide (attached as a knowledge source)
- Include standard protections: limitation of liability, termination for convenience, data
protection provisions
- Flag any unusual terms that require partner review
- Always include a disclaimer that the draft must be reviewed by a qualified solicitor before
use
```

5. Upload your firm's clause library as a knowledge source. This can be a Word document or PDF containing your standard clauses for NDAs, service agreements, employment contracts, and other common documents.
6. Define topics. Topics are pre-built conversation paths that guide the agent. Create three:
   - NDA: "Draft a mutual non-disclosure agreement for use between two UK companies."
   - Service Agreement: "Draft a standard service agreement for IT consultancy services."
   - Employment Contract: "Draft a standard employment contract for a permanent employee in England."
7. Test the agent in the built-in chat panel. Type: "Draft an NDA for a potential acquisition discussion." Review the output.
8. Refine the instructions based on what you see. If the output is too generic, add more specific guidance. If it misses key clauses, list them explicitly.

15.3 Integrating with Microsoft Word

Once the agent works in Copilot Studio, you can make it available in Word:

1. In Copilot Studio, go to the "Channels" or "Integrations" section.
2. Enable the Microsoft Word integration. This adds the agent to Word's Copilot sidebar.
3. In Word, open the Copilot panel. Your Contract Drafting Assistant should appear as an available agent.
4. Test it: open a new document, open Copilot, and type: "Draft a service agreement for a software development project. Include IP ownership, payment terms, and a 30-day termination clause."
5. The agent generates the draft directly in the Word document. You edit, review, and finalise.

15.4 Adding a review checklist step

A good contract drafting agent does not just generate text. It tells you what to check. Add this to your agent's instructions:

```
After generating any draft, include a review checklist at the end:
- Verify all party names and details are correct
- Confirm the governing law and jurisdiction
- Check that the liability cap is appropriate for the transaction size
- Ensure data protection provisions comply with UK GDPR
- Review termination provisions for fairness
- Confirm the signature block is complete
```

This turns the agent from a text generator into a drafting assistant that helps you work faster without skipping the steps that matter.

16 Chapter 12: Due diligence workflow (chaining AI steps)

A client calls on Monday. The acquisition closes in six weeks. Somewhere in the target company's files sit 300 contracts nobody has read properly. Two years ago, that meant hiring a team of juniors and cancelling weekend plans. Today, you can build a tool that does the first pass before lunch.

16.1 The use case

In a typical M&A due diligence exercise, a lawyer receives a data room containing hundreds of contracts. The task is to review each one, extract key terms, flag risks, and summarise the findings in a report. A junior team might spend weeks on this work.

Harvey's Workflows product automates this process using agentic chains: multi-step AI pipelines that classify documents, extract data, flag issues, and generate reports with minimal human intervention. Harvey reports processing over 400,000 agentic queries per day for its clients.

You can replicate about 80% of this capability for free using OpenCode and Gemini 2.5 Pro. The key advantage of Gemini for this work is its one million token context window, which means it can process very long contracts in a single call without losing the beginning of the document by the time it reaches the end.

The trade-off is that you will need to design the workflow yourself and iterate on the prompts. That is what this chapter covers.

16.2 Designing the workflow

A due diligence workflow has five steps:

Step 1: Classify. The AI reads each document and determines what type of contract it is. Employment agreement, lease, service agreement, NDA, loan document, licence. This determines which extraction template to apply in the next step.

Step 2: Extract. For each contract, the AI extracts the key fields that matter for due diligence: parties, effective date, expiration date, renewal terms, termination rights, governing law, liability caps, change-of-control provisions, and any unusual clauses.

Step 3: Flag. The AI identifies risk factors: change-of-control clauses that could be triggered by the acquisition, non-assignment provisions, unlimited liability, unusual termination penalties, or missing key protections.

Step 4: Summarise. The AI produces a one-paragraph summary of each contract, highlighting the key terms and any flagged risks.

Step 5: Output report. All of the above is compiled into a formatted report, organised by contract type, suitable for client delivery.

The technique that makes this work is prompt chaining: breaking one large job into sequential AI calls, where the output of each step becomes the input for the next. Instead of asking the AI to do everything at once (which produces shallow results), you ask it to do one thing well, then feed those results into the next step.

16.3 Build and run

1. Create a project folder and open OpenCode:

```
mkdir due-diligence
cd due-diligence
opencode
```

2. Switch to Plan mode.
3. Paste this prompt:

```
Build a due diligence workflow tool. It should:
- Accept a folder of PDF contracts as input
- For each contract: classify the type, extract key terms, flag risks, and produce a summary
- Key terms to extract: Parties, Effective Date, Expiration Date, Renewal Terms, Termination
Rights, Governing Law, Liability Cap, Change-of-Control Provisions
- Risk flags to identify: change-of-control triggered by acquisition, non-assignment clauses,
unlimited liability, unusual termination penalties, missing key protections
- Compile all results into a formatted report organised by contract type
- Export the report as a Word document (.docx)
- Use Gemini 2.5 Pro via the Gemini API
- Show me your plan first.
```

4. Review the plan carefully. This is a complex build. Check that it includes all five steps: classification, extraction, flagging, summarisation, and report generation. Check that it uses Gemini 2.5 Pro specifically, not a smaller model.
5. Request adjustments:

```
Add a progress indicator that shows which document is being processed and which step it is on.
Also, add a summary page at the beginning of the report with statistics: total contracts
reviewed, number of each contract type, number of high-risk flags.
```

6. Switch to Build mode. This build will take longer than the previous tutorials. OpenCode may need to create multiple files and install several dependencies.
7. Run the app and test it with a small set of sample contracts. Start with five to ten documents. Check each step:
   - Are the contract classifications correct?
   - Are the extracted fields accurate?
   - Are the risk flags appropriate?
   - Is the summary useful?
   - Is the report well-formatted?
8. Iterate. The most common issues at this stage:

```
The classification is wrong for some documents. Add a confidence score: if the model is less
than 80% confident about the contract type, label it as "Uncertain" and include it in a
separate section of the report.
The extraction misses fields that are worded unusually. Add a fallback: if a field cannot be
found, output "Not found" instead of guessing.
```

9. Once the workflow works on a small set, scale up. Try it with 20, then 50 contracts. Watch for performance issues. If processing is slow, ask OpenCode to add parallel processing:

```
Process up to 5 contracts simultaneously to speed up the workflow.
```

The finished tool will not replace a thorough lawyer's review of a data room. It will get you 80% of the way there in a fraction of the time, leaving you to focus on the judgement calls that require a human.

17 Chapter 13: Building a firm prompt library

A senior associate spends twenty minutes crafting the perfect prompt for extracting termination clauses. It works beautifully. Then she leaves the firm, and the prompt leaves with her. Six months later, another associate starts from scratch.

This scenario plays out in firms every day. The fix is not better prompts. It is a system for capturing and sharing them.

17.1 Why every lawyer needs a prompt library

A prompt library is a shared collection of tested, reliable prompts for common legal tasks. It is the equivalent of a firm's precedent bank, but for AI instructions instead of contract clauses.

Legora offers this as a built-in feature: teams can create, share, and collaborate on prompt libraries within the platform. The prompts in this guide are a starting point. A firm prompt library is what turns those starting points into institutional knowledge.

The difference between a good prompt and a great one is iteration. The first version of a prompt produces acceptable output. The tenth version, refined by three different lawyers who each found different weaknesses, produces excellent output. A prompt library captures that refinement so every lawyer in the firm benefits.

17.2 Structure and storage

Organise your prompt library by task category. Here is a structure that works well for most practices:

- **Research:** Prompts for legal research, case analysis, statutory interpretation, and regulatory guidance.
- **Drafting:** Prompts for contracts, letters, memos, pleadings, and client communications.
- **Review:** Prompts for contract review, due diligence, compliance checking, and risk assessment.
- **Client communication:** Prompts for client updates, plain-English explanations of legal concepts, and fee estimates.
- **Compliance:** Prompts for GDPR analysis, anti-money laundering checks, and regulatory filings.

Within each category, use a consistent template format:

```
[Context]: Who is this for? What is the practice area? What jurisdiction?
[Task]: What should the AI do? Be specific.
[Output format]: What should the response look like? Memo, table, bullet points, draft clause?
[Constraints]: What should the AI avoid? What tone should it use? Any mandatory inclusions?
```

Here is an example of a well-structured prompt from the Review category:

```
[Context]: UK commercial law practice. Reviewing a SaaS agreement on behalf of the customer.
[Task]: Identify all clauses that are unfavourable to the customer. For each, explain why it
is unfavourable and suggest alternative wording.
[Output format]: A table with three columns: Clause Reference, Issue, Suggested Alternative.
[Constraints]: Focus on liability, termination, data ownership, and SLA provisions. Use
British legal terminology. Flag any clause that would be unusual in a standard SaaS agreement.
```

Store your prompt library wherever your firm already collaborates. A shared Word document works. A Notion page works better because it is searchable and versioned. A plain Markdown file in a shared folder works if your firm prefers simplicity.

The key is that it must be shared. A prompt library that lives in one lawyer's notebook helps one lawyer. A prompt library that lives where everyone can find it helps the firm.

17.3 Contributing to the library

Establish a simple rule: if you refine a prompt and get a better result, share it. Add it to the library with a note about what you changed and why. Over time, the library becomes one of the firm's most valuable assets: a collection of instructions that reliably produce high-quality legal work with AI assistance.

18 Chapter 14: Security, confidentiality & ethics

A solicitor uploads a client's contract to a free AI API. The contract contains the client's name, the other party's name, the deal terms, and a damages clause that could be worth millions. The API processes it on a server in another country. The data is logged. The client never consented.

No regulator would accept "I did not know" as a defence. Yet this scenario is happening in firms right now, not because lawyers are reckless, but because the tools make it frictionless and the risks are invisible.

18.1 The number one rule: never upload real client data to free public APIs

When you use a free API like Gemini through Google AI Studio or any model through OpenRouter, your data leaves your device and travels to a server you do not control. That server may be in a different jurisdiction. The data may be logged, stored, or used to train future models.

This is not a theoretical risk. It is a direct conflict with your duty of confidentiality.

The rule is simple: **never upload real client documents, real client names, real matter details, or any identifiable client information to a free public API.**

Use fictional documents for testing. Use anonymised data for development. When you need to process real client data, use one of the approaches described in the next section.

Understand the distinction:

- **API calls:** Your data is sent to a third-party server. You are trusting that provider's security, data handling policies, and jurisdiction. Free tiers typically offer fewer privacy guarantees than paid enterprise tiers.
- **Local models:** Your data stays on your device. Nothing is sent to any server. The AI runs on your own hardware. This is the only safe approach for sensitive client matters.

GDPR, the SRA's guidance on technology and AI, and bar ethics codes across jurisdictions all impose obligations on lawyers who use AI. The specific rules vary, but the principle is consistent: you are responsible for protecting client data, regardless of what tool you use to process it.

18.2 Running local models for sensitive matters (Ollama)

Ollama is a free, open-source tool that runs AI models on your own computer. It is available for Mac and Windows. When you use Ollama, your data never leaves your machine.

Installing Ollama takes about five minutes:

1. Go to https://ollama.com and download the installer for your operating system.
2. Run the installer. No command line is needed for the basic setup.
3. Open a terminal and pull a model:

```
ollama pull llama3.3
```

4. Test it:

```
ollama run llama3.3
```

You now have a fully functional AI model running on your device. It is slower than cloud APIs, especially on older hardware, but it is private.

To connect Ollama to OpenCode:

1. Open OpenCode's configuration. Ask OpenCode directly:

```
Add Ollama as a local provider. Use the model llama3.3.
```

2. OpenCode will configure the connection. Once connected, you can select the Ollama provider when starting a new session.
3. Verify it works by starting a new OpenCode session with the Ollama provider and sending a test prompt.

Recommended models for local legal work:

- **Llama 3.3** (70 billion parameters): Good general performance. Requires at least 16GB of RAM, preferably 32GB.
- **Mistral Large**: Strong reasoning capabilities. Higher hardware requirements.
- **Phi-4** (14 billion parameters): Lighter weight, runs on more modest hardware. Good for straightforward tasks.

The quality of local models is lower than the best cloud models. For sensitive client work, that trade-off is worth it. Use local models when confidentiality matters. Use cloud APIs when you are working with fictional or anonymised data and need the best possible output.

18.3 Logging, audit trails & professional responsibility

18.3.1 Why you must review every AI output before use

AI can draft, summarise, extract, compare, and propose. It cannot judge. It does not know your client's commercial objectives, risk tolerance, or the other side's likely position. It does not know which arguments will persuade a particular judge or which clauses a particular client will never accept.

Every output an AI generates is a first draft. You are responsible for verifying that it is accurate, complete, appropriate, and fit for purpose. This is not a new obligation. It is the same obligation you have always had when reviewing work product, whether it was prepared by a junior associate or a paralegal.

The risk with AI is that the output sounds confident and looks polished, which makes it easy to trust without checking. Resist that impulse.

18.3.2 Keeping records of AI-assisted work product

When you use AI to assist with client work, keep a record of:

- Which tool or model was used.
- What input was provided (the prompt and any documents).
- What output was generated.
- What review and editing you performed before the work product was finalised.

This record serves two purposes. First, it demonstrates that you exercised professional judgement rather than simply accepting AI output. Second, it provides an audit trail if a question ever arises about how a particular piece of work was prepared.

You do not need a sophisticated system for this. A simple log in a spreadsheet or document is sufficient. The key is consistency: log every use, every time.

18.3.3 Disclosure obligations to clients and courts

The question of whether you must disclose your use of AI to clients or courts is evolving. Some jurisdictions are beginning to require disclosure of AI use in filed documents. Some clients are beginning to ask.

The safest approach is transparency. If a client asks whether you used AI to prepare a document, tell them. If a court requires disclosure, disclose. Do not present AI-generated work as if it were entirely your own creation, and do not hide the use of AI when asked.

This is an area where the rules are still developing. Keep an eye on guidance from your regulator, your bar association, and your professional indemnity insurer. When in doubt, disclose.

18.4 The bottom line

AI is a tool. Like any tool, it can be used well or poorly, safely or recklessly. The lawyers who benefit most from AI will be those who combine its capabilities with their own judgement, who treat confidentiality as non-negotiable, and who never forget that the work product bears their name.

Build tools. Iterate on prompts. Chain workflows. But never outsource your professional responsibility to a model that does not know your client, does not understand your jurisdiction, and does not face the consequences if something goes wrong.

Note: Hardware specifications and model parameter counts in this chapter are drawn from vendor documentation referenced in the outline. TODO: verify against primary sources before publication.

19 Appendix A: Tool stack summary

This table lists every tool referenced in this guide, what it is for, what it costs, and where to find it.

Tool	Purpose	Cost	URL
Google AI Studio	Gemini API key	Free	aistudio.google.com
OpenRouter	Multi-model API key	Free tier	openrouter.ai
OpenCode	Vibe coding agent (terminal)	Free / OSS	opencode.ai
VS Code	Code editor (optional)	Free	code.visualstudio.com

GitHub Copilot	AI in VS Code	Free tier	github.com
MS Copilot (M365)	Drafting and research in Word/ Teams	M365 licence	copilot.microsoft.com
Copilot Studio	Custom agent builder	Free tier	copilot.microsoft.com/studio
Ollama	Local private LLMs	Free / OSS	ollama.com

20 Appendix B: Prompt templates

These templates are ready to use. Copy them into your prompt library, customise the bracketed fields, and adapt them to your practice.

20.1 Contract Q&A prompt

```
You are a [jurisdiction] solicitor reviewing a [document type].

I will provide the text of the document and a question about it. Respond with:
1. A direct answer to the question
2. The specific clause or section that supports your answer
3. Any related risks or issues the lawyer should be aware of

If the document does not contain enough information to answer the question, say so clearly.

Document: [paste document text]
Question: [your question]
```

20.2 Tabular extraction prompt

```
You are a [jurisdiction] solicitor conducting a review of multiple contracts.

For each contract I provide, extract the following fields and present them in a table:
- Party Names
- Governing Law
- Effective Date
- Expiration Date
- Termination Clause (summarise in 2 sentences)
- Liability Cap
- Notice Period
- Renewal Terms

If a field cannot be found in the contract, write "Not found" rather than guessing.

Contract: [paste contract text]
```

20.3 Legal research memo prompt

```
You are a [jurisdiction] solicitor. Draft a research memo on the following question.

Jurisdiction: [specify jurisdiction]
Question: [your question]

Structure the memo as follows:
1. Question Presented (restate the question)
2. Brief Answer (2-3 sentences)
3. Discussion (detailed analysis with references to case law and statutes)
4. Conclusion (practical advice for the solicitor handling this matter)

Include a disclaimer that all citations must be verified against primary sources before
reliance.
```

20.4 Risk flagging prompt

```
You are a [jurisdiction] solicitor reviewing a [document type] on behalf of [client type].
```

```
Identify all clauses that pose a risk to the client. For each risk:
1. Quote the relevant clause
2. Explain why it is risky
3. Rate the risk as High, Medium, or Low
4. Suggest alternative wording that would better protect the client

Focus on: liability provisions, termination rights, indemnification, data protection, and any
unusual or non-standard terms.

Document: [paste document text]
```

20.5 Redlining / markup prompt

```
You are a [jurisdiction] solicitor. I will provide the current version of a clause and the
client's requested changes.

Provide:
1. A tracked-changes version showing the proposed amendments
2. A brief explanation of each change
3. Any risks introduced by the proposed changes
4. Any additional protections that should be considered

Current clause: [paste current text]
Requested changes: [describe changes]
```

21 Appendix C: Glossary for lawyers

This glossary defines every technical term used in this guide. Terms are explained in plain English, not technical jargon.

21.1 API (Application Programming Interface)

A way for two software programs to talk to each other. When you use the Gemini API, your app sends a request to Google's servers and receives a response. Think of it as a messenger that carries your question to the AI and brings the answer back.

21.2 API key

A unique code that identifies you to an API provider. It works like a password. Anyone who has your API key can use your account, potentially running up charges or accessing your data. Store it securely and never share it in code, email, or chat.

21.3 Token

A unit of text that AI models process. One token is roughly one word or a large part of a word. "Contract" is one token. "Indemnification" might be two or three. Every model has a limit on how many tokens it can process in a single request.

21.4 Context window

The maximum number of tokens a model can consider at once. A model with a 100,000 token context window can read about 75,000 words in a single request, roughly 300 pages. Gemini 2.5 Pro offers a one million token context window, which means it can process an entire folder of contracts in one call. This matters for legal work because long documents lose coherence when the model cannot see the whole thing at once.

21.5 RAG (Retrieval-Augmented Generation)

A technique where the AI searches for relevant documents or data before generating an answer. Instead of relying only on what it learned during training, it retrieves up-to-date information from a database, a website, or a set of documents you provide. This improves accuracy and reduces hallucination.

21.6 Agent / agentic workflow

An AI agent is a system that can take actions, not just answer questions. It can search the web, read files, run code, and make decisions. An agentic workflow chains multiple actions together: the AI classifies a document, then extracts data, then flags risks, then generates a report. Each step builds on the previous one.

21.7 LLM (Large Language Model)

The technical term for the type of AI used in this guide. An LLM is a model trained on vast amounts of text that can generate human-like responses. GPT-4, Claude, Gemini, Llama, and Mistral are all LLMs. They differ in training data, size, capabilities, and cost.

21.8 Vibe coding

The practice of building software by describing what you want in plain English and letting AI generate the code. You do not write code. You describe requirements, review plans, and iterate on results. The AI handles the technical implementation.

21.9 MCP (Model Context Protocol)

A standard way for AI models to connect to external tools and data sources. MCP lets an AI assistant read files from your computer, query a database, or interact with a legal research platform. It is the plumbing that connects AI models to the outside world.

21.10 Fine-tuning vs. prompting

Fine-tuning means training a model on your specific data so it learns your firm's style, terminology, and precedents. It is expensive and technically complex. Prompting means giving the model instructions and context in your request. It is free and immediate. For most lawyers, prompting is the right approach. Fine-tuning is for organisations with large, consistent datasets and technical resources.

22 Appendix D: Troubleshooting FAQ

22.1 "My API key is not working"

Check these in order:

1. Did you copy the full key? Keys are long. Make sure you did not miss characters at the beginning or end.
2. Is the key stored in a `.env` file? The app needs to read it from the environment, not from a comment or a separate document.
3. Did you restart the app after adding the key? Environment variables are loaded when the app starts.
4. Has the key expired? Some providers rotate keys. Generate a new one from the provider's dashboard.
5. Have you exceeded your free quota? Check your usage on the provider's website. Free tiers have daily or monthly limits.

22.2 "The app built but does not do what I asked"

This is the most common issue and it is almost always a prompt problem, not a code problem.

1. Re-read your prompt. Was it specific enough? "Build a contract review tool" is vague. "Build a web app that accepts PDF uploads, extracts the termination clause, and flags if the notice period is less than 30 days" is specific.
2. Check the plan. Did you review the plan before switching to Build mode? If the plan was wrong, the build will be wrong. Undo and fix the plan first.
3. Iterate with precision. Instead of "fix it," say "the upload button only accepts PDF files. I also need it to accept Word documents (.docx)."
4. Check the model. Some tasks require a more capable model. If Gemini Flash is producing shallow results, switch to Gemini 2.5 Pro or DeepSeek R1.

22.3 "I am worried about client confidentiality"

You should be. Here is the safe approach:

1. For development and testing, use only fictional documents. Invent a company, write a sample contract, and test with that.
2. For real client work, use Ollama to run models locally. Your data stays on your device.
3. If you must use a cloud API for client work, use a paid enterprise tier that offers data processing agreements and does not use your data for training. Google, Anthropic, and OpenAI all offer enterprise options with stronger privacy commitments than their free tiers.
4. Never paste client names, matter details, or identifiable information into a free chat interface or API.

22.4 "How do I share this tool with my colleagues?"

There are several approaches, depending on technical comfort:

1. **Share the project folder.** Copy the folder to a shared drive. Each colleague installs the dependencies (`npm install`) and runs the app locally. They will need their own API key in a `.env` file.
2. **Deploy to the web.** Ask OpenCode to deploy the app to a free hosting service like Vercel or Netlify. This makes it accessible via a URL. Be careful: a web-deployed app that uses a cloud API means data leaves your firm's network.
3. **Use Copilot Studio.** If the tool is a Copilot Studio agent, share it through your Microsoft 365 admin centre. Colleagues access it through Teams or Word.
4. **Docker container.** For more technical teams, ask OpenCode to create a Docker image. This packages the app and all its dependencies into a single file that runs the same way on any machine.

22.5 "The AI keeps hallucinating legal citations"

Hallucination is the most dangerous behaviour for legal work. Here is how to reduce it:

1. Add explicit instructions: "Only cite cases and statutes that you are confident exist. If you are unsure, say so rather than guessing."
2. Use RAG: connect the model to a legal database so it retrieves real sources instead of generating from memory.
3. Add a verification step: ask the model to provide a source URL or paragraph number for every citation. Then check them.
4. Use reasoning models like DeepSeek R1, which are less prone to confident fabrication than fast models.
5. Always, without exception, verify every citation against a primary source before relying on it in client work.